

Riemann_Lab — Méthode scripts post-audit et remarques matériel

Document de méthode généré après la checklist système Linux avant migration v1 — exigences Phase C. Il récapitule les scripts et commandes ajoutés depuis cette étape, puis rassemble en fin de rapport les remarques détaillées sur le matériel zeta-lab.

Sources utilisées : résumé d'archive `assistant_pack_audit_resume_global_v2.txt`, pack texte `assistant_pack_audit_details_v2.txt`, diagnostic système précédemment consolidé et scripts créés dans la conversation.

1. Objectif de la méthode

- Éviter de transmettre une archive brute de 12 Go et travailler plutôt avec des résumés texte ciblés.
- Conserver l'archive complète comme sauvegarde locale, mais extraire seulement les fichiers utiles à l'analyse.
- Standardiser le flux : audit complet → résumé des archives → extraction texte → analyse assistant → mini-audit Git complémentaire.
- Préparer la migration hprzeta-lab sans commandes destructrices, en gardant `/home/riemann/projet_zeta` et `/mnt/data` protégés.

2. Problème rencontré

Le script `audit_systeme_hprzeta_avant_reinstall.sh` a produit une archive très volumineuse `hprzeta_system_audit_2026-05-27_01-40-50.tar.gz` d'environ 12 Go. Ce volume est adapté à une sauvegarde complète, mais pas à une analyse directe dans l'assistant. Le premier résumé a aussi montré une faute d'option `--sah256` au lieu de `--sha256`, corrigée ensuite avec le mode `--auto`.

Point de méthode : ne jamais ouvrir manuellement tous les ZIP/TAR. Toujours générer un index texte avant d'extraire.

3. Script 1 — `make_assistant_archive_summary.sh`

Ce script a été créé pour lister le contenu d'un ou plusieurs `.zip/.tar/.tar.gz` sans décompresser toute l'archive. Il produit un fichier texte global exploitable par l'assistant.

Élément	Rôle
Entrée	Une ou plusieurs archives <code>.zip</code> , <code>.tar</code> , <code>.tar.gz</code> , <code>.tgz</code> , <code>.tar.xz</code> , etc.
Sortie	<code>assistant_pack_audit_resume_global.txt</code> ou nom choisi avec <code>-o</code>
Options clés	<code>--auto</code> , <code>--sha256</code> , <code>--max-lines</code> , <code>-o</code>
Usage recommandé	Mode automatique avec <code>sha256</code> et limitation de lignes
Bénéfice	Permet de savoir quoi extraire sans envoyer ni ouvrir une archive énorme

Commande recommandée :

```
cd /home/riemann/projet_zeta/scripts
./make_assistant_archive_summary.sh --auto --sha256 --max-lines 8000 -o assistant_pack_audit_resume_global_v2.txt
```

Variante légère si `tar -tf` ou `sha256sum` prend trop de temps :

```
./make_assistant_archive_summary.sh --auto --max-lines 1000 -o assistant_pack_audit_resume_global_light.txt
```

4. Résultat obtenu avec le résumé v2

- Le résumé v2 a correctement détecté une seule archive `tar.gz` : `./hprzeta_system_audit_2026-05-27_01-40-50.tar.gz`.
- La taille de l'archive est d'environ 12 Go.
- Le SHA256 obtenu est `89426b5d1a956b0ec5729d30a25c61b0918756dbc98b565220e20878f928d081`.
- La structure interne contient `services`, `packages`, `manifests`, `checksums`, `python`, `configs`, `hardware`, `system`, `projects`, `git` et `logs`.
- La bonne conclusion était : ne pas envoyer l'archive de 12 Go, mais extraire un pack texte ciblé.

5. Script/commande 2 — extraction texte ciblée depuis la grosse archive

Après lecture du résumé v2, la méthode retenue consiste à extraire seulement les dossiers et fichiers texte nécessaires à l'audit : manifests, system, hardware, python, configs, services, git, checksums et inventaires projets.

```
cd /home/riemann/projet_zeta/scripts
mkdir -p audit_extract_text_v2
```

```
tar -xzf hprzeta_system_audit_2026-05-27_01-40-50.tar.gz -C audit_extract_text_v2 audit_2026-05-27_01-40-50/manifests audit_2026-05-27_01-40-50/
```

Cette extraction ne récupère pas les gros snapshots internes .tar.gz : elle prépare seulement un paquet d'analyse léger.

6. Script/commande 3 — génération du fichier assistant_pack_audit_details_v2.txt

La seconde étape transforme le dossier extrait en un seul fichier texte avec la liste des fichiers et les premières lignes de chaque fichier texte.

```
cd /home/riemann/projet_zeta/scripts/audit_extract_text_v2
OUT="./assistant_pack_audit_details_v2.txt"
```

```
{
echo "===== LISTE DES FICHIERS EXTRAITS ====="
find . -type f | sort
echo

echo "===== CONTENU DES FICHIERS TEXTE ====="
find . -type f \( -name "*.txt" -o -name "*.md" -o -name "*.list" -o -name "*.sources" \) | sort | while read -r f; do
echo
echo "===== FILE: $f"
echo "===== "
sed -n '1,300p' "$f"
done
} > "$OUT"
```

Ce fichier a ensuite permis l'analyse du matériel, d'Ollama, de Python, de /mnt/data et des inventaires projet.

7. Script/commande 4 — mini-audit Git complémentaire

La partie Git de l'audit initial a échoué à cause d'une erreur sed. Il faut donc compléter avec un mini-audit Git indépendant.

```
cd /home/riemann/projet_zeta
```

```
{
echo "===== DATE ====="
date
echo

echo "===== GIT BRANCH -VV ====="
git branch -vv
echo

echo "===== GIT STATUS ====="
git status
echo

echo "===== GIT REMOTE -V ====="
git remote -v
echo

echo "===== GIT LOG LAST 20 ====="
git log --oneline --decorate --graph --all -20
echo

echo "===== BRANCHES FILE DIFF IA..C ====="
git diff --name-status Riemann_Lab_IA..Riemann_Lab_C
echo

echo "===== FILES IN RIEMANN_LAB_C PHASE C ====="
git ls-tree -r --name-only Riemann_Lab_C | grep -E "c_modules|compute_zeros_v4|compute_zeros_v3|illinois|benchmark|optimisation" || true
} > ~/mini_audit_git_riemann_lab.txt
```

8. Chaîne de méthode complète recommandée

Étape	Commande ou action	Produit attendu
1	Lancer <code>audit_systeme_hprzeta_avant_reinstall.sh</code>	Archive complète locale tar.gz/zip
2	<code>make_assistant_archive_summary.sh --auto --sha256</code>	Résumé des archives + SHA256
3	Extraire seulement manifests/system/hardware/python/configs/services/projects/git/checksums	Dossier <code>audit_extract_text_v2</code>
4	Créer <code>assistant_pack_audit_details_v2.txt</code>	Fichier texte analysable
5	Mini-audit Git séparé	<code>mini_audit_git_riemann_lab.txt</code>
6	Analyse assistant	PDF audit + scripts bootstrap + plan migration

9. Règles de sécurité et de reproductibilité

- Ne jamais supprimer ou déplacer `/home/riemann/projet_zeta` avant sauvegarde et hash.
- Ne jamais envoyer l'archive de 12 Go si un résumé texte suffit.
- Toujours calculer ou conserver les SHA256 pour les archives principales.
- Toujours distinguer archive complète, extrait texte, snapshot léger et repo actif.
- Ne pas réutiliser aveuglément `zeta_env` : le conserver comme référence, puis reconstruire des environnements propres.
- Corriger le script audit Git : l'erreur `sed` a empêché la capture correcte de `local_git_repos` et `repo_status`.

10. Scripts à prévoir ensuite

- `backup_all_branches.sh` : sauvegarde branches main, Riemann_Lab_IA, Riemann_Lab_C, Riemann_Lab_Test avec git bundle et tar.gz.
- `restore_branch_C.sh` : restaure la branche C et prépare les dépendances C/libmpfr.
- `build_phase_c.sh` : compile c_modules, produit `illinois_mpfr.so`, lance `test_illinois.py`.
- `validate_run_v3.py` : contrôle CSV/log/monotonie/doublons/LMFDB/Turing.
- `bootstrap_hprzeta_ubuntu24.sh` : installe la base propre Ubuntu 24.04 pour hprzeta-lab.
- `rag_ingest_riemann_lab_c.py` : indexe les fichiers validés et les spécifications Phase C sans mélanger legacy et brouillons.

Annexe — Remarques détaillées sur le matériel zeta-lab

Cette annexe reprend le diagnostic matériel issu du pack assistant_pack_audit_details_v2.txt et doit être placée en fin de rapport comme demandé.

A. Identité système

- Machine : zeta-lab.
- Système : Ubuntu 24.04.4 LTS Noble, noyau 6.17.0-29-generic, architecture x86_64.
- Locale : fr_FR.UTF-8.
- Matériel : ASUS UX510UWK, firmware UX510UWK.300 datant de 2016.

B. Processeur

- CPU : Intel Core i7-7500U @ 2.70 GHz.
- Topologie : 2 cœurs physiques, 4 threads logiques, 1 socket.
- Fréquence max : 3.5 GHz ; fréquence min : 400 MHz.
- Instructions utiles : AVX, AVX2, FMA, SSE4.1/SSE4.2, AES, BMI1/BMI2.
- Conclusion : CPU correct pour calculs zêta modérés et parallélisme 4 workers, mais pas une station lourde. Les runs longs doivent rester monitorés.

C. Mémoire et swap

- RAM : environ 7.6 GiB.
- État au moment de l’audit : environ 3.8 GiB utilisés, 184 MiB libres, environ 3.8 GiB disponibles via cache.
- Swap : environ 15–16 GiB, avec /mnt/data/swapfile et une partition swap.
- Conclusion : configuration viable, mais RAM limitée. Éviter les notebooks trop chargés, les embeddings massifs en mémoire et les modèles IA trop gros.

D. Stockage et partitionnement

Partition	Taille	Usage	Commentaire
/	≈74.5G	≈54% utilisé	Taille correcte pour OS et paquets
/boot/efi	≈1G	≈1% utilisé	OK
/home	≈186.3G	≈18% utilisé	Espace de travail utilisateur correct
/mnt/data	≈662.2G	≈5% utilisé	Très bon volume pour modèles, exports, logs, datasets
swap	≈7.5G + swapfile 16G	quasi libre	Bon filet de sécurité avec 8 Go RAM

- Le montage /mnt/data en ext4 avec noatime est adapté aux modèles Ollama, gros exports, snapshots et données validées.
- La séparation /, /home et /mnt/data est saine. Il faut la conserver ou la reproduire si réinstallation complète.

E. GPU NVIDIA

- GPU : NVIDIA GeForce GTX 960M, architecture GM107M.
- VRAM : 4 Go.
- Driver : NVIDIA 535.309.01.
- CUDA visible par nvidia-smi : 12.2.
- Modules chargés : nvidia, nvidia_uvm, nvidia_drm, nvidia_modeset.
- État audit : 42°C, environ 460 MiB VRAM utilisés, GPU visible et actif.
- Conclusion : GPU utilisable pour CuPy léger, affichage, Ollama léger, mais pas pour gros LLM. Privilégier phi3:mini, qwen3:4b, mathstral avec prudence, éviter modèles lourds.

F. Ollama et modèles IA

- Service ollama actif et enabled, lancé via systemd.

- Modèles présents : phi3:mini 2.2 Go, qwen3:4b 2.5 Go, mathstral:latest 4.1 Go, deepseek-coder:6.7b 3.8 Go.
- Variables : OLLAMA_MODELS=/mnt/data/models_ia/ollama et OLLAMA_CUDA=1.
- Conclusion : implantation correcte. Les modèles sont sur /mnt/data, ce qui protège /home et /. Le choix de petits modèles est cohérent avec 4 Go VRAM.

G. Python et environnement zeta_env

- Python : 3.12.3 ; pip : 24.0.
- Venv principal : /home/riemann/projet_zeta/zeta_env.
- Environnement très chargé : calcul scientifique, IA, Jupyter, CUDA, RAG, dev tools, profiling.
- Conclusion : zeta_env est précieux comme archive et référence, mais ne doit pas être reconstruit tel quel. Préférer plusieurs environnements : minimal, dev, ia, gpu-optionnel.

H. Risques identifiés

- Erreur sed dans l'audit Git : local_git_repos et repo_status ne sont pas fiables. Mini-audit Git obligatoire.
- credential.helper=store dans Git : à remplacer par une méthode plus sûre lors de la migration.
- Multiples copies Riemann_Lab dans Documents/saved, hprzeta-import et projet_zeta : risque de confusion et de doublons.
- Archive audit de 12 Go : excellente sauvegarde, mais à ne pas manipuler constamment. Travailler avec extraits ciblés.
- RAM limitée : surveiller Jupyter, Ollama, embeddings et batchs GPU.

I. Verdict matériel

Le matériel est cohérent avec le projet hprzeta-lab si la méthode reste sobre : calcul CPU optimisé, GPU utilisé en option, IA locale légère, stockage /mnt/data pour modèles et données, recovery robuste et environnements Python séparés. La machine ne doit pas être traitée comme une station GPU moderne, mais elle est très exploitable pour les runs Riemann_Lab jusqu'à T=10000, la documentation, les visualisations, la préparation RAG légère et les développements Phase C.